

OAuth / OpenID Connect Integration Guide

Technical reference for client engineering teams

v1.3 · May 2026

- No SDK to install
- Standard OIDC endpoints
- Zero changes to your application logic
- Fault-tolerant, HA-ready

TABLE OF CONTENTS

TABLE OF CONTENTS	2
1. Overview	4
2. What Your Team Adds — The Complete Picture	4
3. Step 1 — Client Registration	5
Grant Type Guidance	5
4. Prerequisite — End-User Device Enrollment	6
Two Distinct Concepts	6
Device Enrollment API — POST /reg-device	6
Per-User Device Limits	6
5. Step 2 — Authorization Request	8
Redirect URL	8
Parameters	8
6. Step 3 — Handle the Callback	9
Callback URL	9
Callback Handler (Pseudocode)	9
7. Step 4 — Token Exchange	10
Token Endpoint Request	10
Token Endpoint Response	10
ID Token Claims	10
8. OIDC Discovery & Standard Endpoints	12
JWKS Key Structure	12
9. Invysta Configured as MFA	13
How the Flow Works	13
/auth/external Parameters	15
Direct Flow vs. MFA Flow	16
10. Fault Tolerance & High Availability	17
Token TTLs (Configurable)	17
11. Security Model	18

1. Overview

Invysta operates as a managed **OAuth 2.0 / OpenID Connect (OIDC) Provider**. Your application delegates authentication to Invysta exactly as it would to any enterprise identity provider (Okta, Auth0, Azure AD). The only difference is what happens inside the login interaction: Invysta silently layers its device-fingerprint verification on top — completely transparent to your codebase.

What you are integrating with: Invysta runs a hardened oidc-provider node, accessible at your dedicated tenant URL (e.g. <https://auth.invysta-technical.com>). Your application registers as an OIDC client. All device-fingerprint logic lives entirely server-side at Invysta — you simply redirect users there for login and receive a standard token in return.

2. What Your Team Adds — The Complete Picture

There are **three integration points**, each mapping directly to a standard OIDC pattern your team already knows.

1

Register your application as an OIDC client

Invysta provisions you a `client_id` (and optionally a `client_secret`). You supply your `redirect_uri` and preferred grant type. One-time, five-minute configuration — no code required.

Supported grant types: `authorization_code` (recommended) · `implicit` (legacy/SPA only)

2

Send users to the Invysta authorization endpoint

When a user initiates login, redirect their browser to the Invysta `/auth` endpoint. Invysta handles the full interaction — credential check, device fingerprint verification, OTP if required — and redirects back with an authorization code.

3

Exchange the code for tokens at the token endpoint

Your backend posts the authorization code to Invysta's `/token` endpoint and receives a standard JWT access token and OIDC `id_token`. Use the `id_token` to establish your session. Done.

3. Step 1 — Client Registration

Invysta provisions your client configuration during onboarding. No API call needed from your side.

Field	Your Value	Notes
<code>client_id</code>	Assigned by Invysta	Unique identifier for your application
<code>client_secret</code>	Assigned by Invysta — server-side flows only	Omit for public/SPA clients (token_endpoint_auth_method: none)
<code>redirect_uris</code>	You provide your callback URL(s)	e.g. <code>https://yourapp.com/auth/callback</code>
<code>grant_types</code>	<code>authorization_code</code> ← RECOMMENDED	See grant type guidance below
<code>response_types</code>	<code>code</code>	Use <code>id_token</code> only for legacy implicit flow
<code>post_logout_redirect_uris</code>	Optional — you provide	Where to send users after logout

Grant Type Guidance

authorization_code is the recommended grant type for all integrations. The authorization code is exchanged server-side for tokens — credentials never pass through the browser. Two variants exist depending on your application type:

App Type	grant_types	client_secret	Notes
Server-side (Node, Java, .NET, Python)	<code>authorization_code</code>	Required	Most secure. Secret stays on your server.
SPA or mobile — no backend secret	<code>authorization_code</code>	Omit — set token_endpoint_auth_method: none	No secret issued or required.
Legacy SPA	<code>implicit</code>	N/A	Tokens in browser URL. No refresh. Avoid.

Nothing to deploy. This configuration lives entirely within the Invysta service. Your team provides the redirect URIs; Invysta provisions the rest.

4. Prerequisite — End-User Device Enrollment

The OIDC client registration (Section 3) is a **one-time configuration** performed by your IT or engineering team. It is separate from, and independent of, device enrollment.

Before any end user can authenticate through Invysta, their device must be enrolled. This is a one-time action per device, performed by the user via the Invysta mobile app. Once enrolled, the device's Anonymous Access Key (AAK) and fingerprint identifiers are registered against that user's account — authentication is then silent and automatic on every subsequent login.

All device fingerprint data is encrypted by the Invysta mobile app using the Invysta service public key before it is transmitted. This end-to-end encryption ensures that fingerprint components and AAK values are never exposed in plaintext to any intermediary, including your platform infrastructure. Only the Invysta service, holding the corresponding private key, can decrypt and process the payload.

The Invysta mobile app retrieves the encryption public key automatically at startup from the Invysta JWKS endpoint. The key is identified by a `kid` (key ID) and carries `"use": "enc"` to distinguish it from the JWT signing key (`"use": "sig"`). The app caches the key locally and re-fetches it automatically when the `kid` changes, ensuring transparent key rotation with no manual coordination required.

Two Distinct Concepts

Concept	Who Does It	When	How Often
OIDC client registration	Your IT / engineering team	During onboarding with Invysta	Once per application
Device enrollment	Each end user	Before their first login	Once per device, up to licensed limit

Device Enrollment API — POST /reg-device

Called by the Invysta mobile app on the user's device. Your application does not call this endpoint. The payload is encrypted by the mobile app before transmission — the Invysta service decrypts and processes it server-side.

```
POST https://auth.invysta-technical.com/reg-device
Content-Type: application/json
```

A 201 Created response confirms the device is enrolled and ready to authenticate.

Per-User Device Limits

Each user account carries a `maxAuthorizedDevices` value set by your Invysta license. Attempting to enroll beyond that limit returns a 403 Forbidden. Device limits are adjustable per user or per company tier by your Invysta account manager.

Login sequence dependency: When a user reaches the Invysta login page, Invysta internally matches the login session (uid + nonce) against the user's registered device fingerprint via POST `/reg-login`. If no enrolled device is found for that user, authentication is denied. **Device enrollment must precede first login.**

5. Step 2 — Authorization Request

When a user clicks "Sign in", redirect their browser to the Invysta /auth endpoint. Standard OAuth 2.0 Authorization Code redirect — no Invysta-specific library needed.

Redirect URL

```
GET https://auth.invysta-technical.com/auth
?client_id=your-app-client-id
&response_type=code
&scope=openid email profile
&redirect_uri=https://yourapp.com/auth/callback
&state=random_csrf_token
&nonce=random_nonce_value
```

Parameters

Parameter	Required	Description
<code>client_id</code>	Required	Your registered client ID from Step 1
<code>response_type</code>	Required	Must be code for authorization code flow
<code>scope</code>	Required	openid is mandatory. Add email and profile for user claims.
<code>redirect_uri</code>	Required	Must exactly match a registered redirect URI
<code>state</code>	Required	CSRF protection. Generate a random value; verify on callback.
<code>nonce</code>	Recommended	Replay attack protection. Echoed in the id_token.
<code>login_hint</code>	Optional	Pre-fill the user's email on the Invysta login page
<code>prompt</code>	Optional	login forces re-authentication even if a session exists

What happens next (Invysta-side, invisible to your code): Invysta validates credentials, verifies the device fingerprint via AAK, and if all checks pass, redirects back to your `redirect_uri` with a short-lived authorization code and your state value. Device fingerprint data exchanged during this interaction is end-to-end encrypted between the Invysta client app and the Invysta service — it is never exposed in plaintext to your platform or any intermediary.

6. Step 3 — Handle the Callback

Invysta redirects back to your `redirect_uri` with two query parameters. Validate state, then exchange the code.

Callback URL

```
GET https://yourapp.com/auth/callback
?code=SpLx10BeZQQYbYS6WxSbIA // one-use authorization code (10 min TTL)
&state=random_csrf_token // verify this matches what you sent
```

Callback Handler (Pseudocode)

```
// 1. Verify state to prevent CSRF
if (query.state !== session.oauthState) throw "Invalid state";

// 2. Exchange code for tokens -- server-side POST to Invysta
const tokens = await exchangeCode(query.code);

// 3. Validate id_token and establish session
const user = validateAndDecode(tokens.id_token);
session.userId = user.sub;
```

7. Step 4 — Token Exchange

Your **backend server** POSTs the authorization code to Invysta's token endpoint. Server-to-server only — never from the browser.

Token Endpoint Request

```
POST https://auth.invysta-technical.com/token
Content-Type: application/x-www-form-urlencoded

grant_type=authorization_code
&code;=Splxl0BeZQQYbYS6WxSbIA
&client_id=your-app-client-id
&client_secret=your-client-secret // omit if token_endpoint_auth_method: none
&redirect_uri=https://yourapp.com/auth/callback
```

Token Endpoint Response

```
{
  "access_token": "eyJhbGciOiJSUzI1NiJ9...", // JWT, 1hr TTL
  "token_type": "Bearer",
  "expires_in": 3600,
  "id_token": "eyJhbGciOiJSUzI1NiJ9...", // OIDC identity token, 1hr TTL
  "refresh_token": "..." // 30-day TTL (if requested)
}
```

ID Token Claims

Claim	Description
<code>sub</code>	Unique user identifier — stable across sessions
<code>email</code>	Verified user email (if email scope was requested)
<code>nonce</code>	Echoed from your authorization request — validate this matches
<code>iss</code>	Issuer — your Invysta provider URL
<code>aud</code>	Your <code>client_id</code> — validate this matches

Claim	Description
exp	Expiry timestamp

Token validation: Invysta publishes its public keys at <https://auth.invysta-technical.com/jwks> (standard JWKS endpoint). The endpoint serves two key types: the JWT signing key ("use": "sig") used by your application to validate tokens, and the payload encryption key ("use": "enc") used by the Invysta client app to encrypt device fingerprint data end-to-end. Use any OIDC-compliant library (openid-client, jose, Passport.js, Spring Security, etc.) to validate token signatures automatically.

8. OIDC Discovery & Standard Endpoints

Invysta publishes a standard OIDC discovery document. Any compliant OIDC library can auto-configure from this URL alone.

```
// Auto-discovery (recommended)
GET https://auth.invysta-technical.com/.well-known/openid-configuration

// Individual endpoints
Authorization: https://auth.invysta-technical.com/auth
Token: https://auth.invysta-technical.com/token
JWKS: https://auth.invysta-technical.com/jwks // signing + encryption keys
Introspection: https://auth.invysta-technical.com/token/introspection
Revocation: https://auth.invysta-technical.com/token/revocation
End session: https://auth.invysta-technical.com/session/end
```

JWKS Key Structure

The JWKS endpoint serves two distinct keys. Both are returned in a single response and are distinguished by their use and kid fields:

```
{
  "keys": [
    {
      "kty": "RSA",
      "use": "sig", // JWT signing key – used by your app to validate tokens
      "kid": "sig-2026-05", // key ID – rotated periodically
      "alg": "RS256",
      "n": "...", "e": "..." // RSA public key components
    },
    {
      "kty": "RSA",
      "use": "enc", // payload encryption key – used by the Invysta client app
      "kid": "enc-2026-05", // key ID – included in encrypted envelope for rotation
      "alg": "RSA-OAEP",
      "n": "...", "e": "..." // RSA public key components
    }
  ]
}
```

Key rotation: When Invysta rotates either key, the `kid` value changes. The Invysta client app detects the new `kid` on its next JWKS fetch and re-caches the updated key automatically. Your application's OIDC library handles signing key rotation the same way. No manual coordination or deployment is required on either side.

9. Invysta Configured as MFA

Invysta can operate as a pure second factor behind your existing authentication system. Your identity provider handles factor one — password, SSO, biometric, whatever you already have. Invysta adds factor two: device fingerprint verification via Anonymous Access Key (AAK). Your application receives a standard OIDC token at the end, exactly as in the direct flow.

Nothing in your existing auth system changes. The only addition is a single redirect call from your auth system to Invysta after credential validation succeeds.

Why this MFA is stronger than TOTP or SMS codes: The second factor is the device identity itself — a cryptographic fingerprint that cannot be phished, forwarded, or intercepted. A one-time code can be stolen in transit; a device fingerprint cannot be replayed from a different device.

How the Flow Works

1

User submits credentials to your system

Your existing auth system validates the password, SSO token, or whatever credential you manage. Invysta is not involved at this stage. This is factor one.

2

Your system hands off to Invysta

On successful credential validation, your auth system redirects the user's browser to the Invysta /auth/external endpoint. This is a browser redirect — not a server-to-server API call. It tells Invysta: 'this user has passed factor one; now verify their device.' This is the only integration touch required on your side.

```
GET https://auth.invysta-technical.com/auth/external
?email=user@yourcompany.com
&provider;=your-idp-name
&client_id=your-registered-client-id
&token;= // your auth token -- passed for audit trail
```

3

Invysta issues a device authentication challenge

Invysta accepts the handoff and internally constructs an OIDC authorization session for the user. It then issues a device authentication challenge to the Invysta client app on the user's device. This exchange is entirely between the Invysta client app and the Invysta service — your host system is not involved and does not see this interaction.

The challenge prompts the client app to collect the current device fingerprint, encrypt it end-to-end using the Invysta service public key, and return the ciphertext directly to the Invysta service. Your platform never sees the fingerprint data.

4

Invysta verifies the device fingerprint

The Invysta service decrypts the payload using its private key and verifies the device fingerprint against the user's enrolled Anonymous Access Key (AAK). This is factor two. Invysta then communicates the outcome to your system via a browser redirect — not a direct API response. See Step 5.

5

Invysta redirects the outcome to your system

Invysta redirects the user's browser to your registered `redirect_uri` carrying either a code (success) or an error (failure). Invysta never sends a raw 401/403 directly to your backend — the communication channel throughout is browser redirects. Your callback handler must check which was received.

Success — browser redirected to your callback with an authorization code

```
GET https://yourapp.com/auth/callback
?code=SpLxl0BeZQQYbYS6WxSbIA // exchange this at /token for the OIDC token
&state;=random_csrf_token
```

Failure — browser redirected to your callback with an error parameter

```
GET https://yourapp.com/auth/callback
?error=access_denied
&error_description=Device+not+recognized
&state;=random_csrf_token
```

Your callback handler checks for the error parameter and responds accordingly — show the user a message, log the failure, or retry. No OIDC token is issued on a failed device check.

Key point: The GET to `/auth/external` (Step 2) is the handoff from your system to Invysta — it is not the device challenge. Invysta issues the device authentication challenge internally in Step 3, directly to the Invysta client app on the user's device. Your host system only participates in Steps 1, 2, and 5.

`/auth/external` Parameters

Parameter	Required	Description
<code>email</code>	Required	The authenticated user's email. Must match their Invysta-enrolled account.
<code>provider</code>	Required	Label identifying your auth system (e.g. okta, azure-ad, internal). Used for audit logging.
<code>client_id</code>	Required	Your registered Invysta client ID. Must match an <code>EXTERNAL_AUTH_PROVIDERS</code> entry configured at onboarding.
<code>token</code>	Optional	A JWT from your identity provider. Audit trail only. Invysta does not re-validate it.

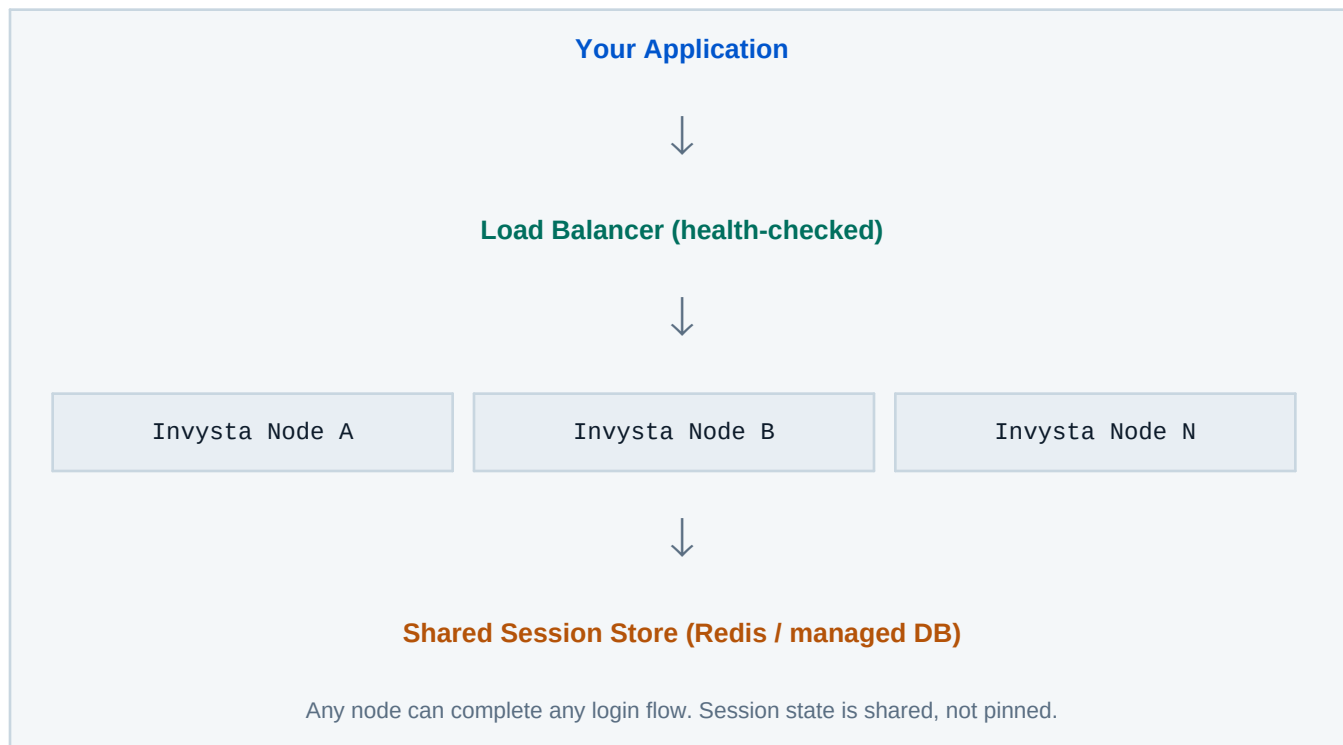
Onboarding requirement: For this flow, Invysta must be configured with your provider entry in EXTERNAL_AUTH_PROVIDERS during tenant setup. One-time configuration handled by Invysta — no code changes on your side.

Direct Flow vs. MFA Flow

	Direct Flow	MFA Flow
Factor one	Invysta (email + password)	Your existing auth system
Factor two	Invysta device fingerprint	Invysta device fingerprint
Token returned	Standard OIDC JWT	Standard OIDC JWT
Changes to your app	OIDC redirect + callback	One browser redirect after credential validation (Step 2). Steps 3–4 handled entirely by Invysta.
Existing auth system	Replaced by Invysta	Unchanged — runs as-is

10. Fault Tolerance & High Availability

Invysta is architected as a managed, multi-node service. Your application is never dependent on a single point of failure, and a failover event is entirely transparent to your codebase.



Token TTLs (Configurable)

Token Type	Default TTL
Authorization Code	10 minutes (one-use)
Access Token	1 hour
ID Token	1 hour
Refresh Token	30 days

Impact on your service if Invysta is unreachable: Users with valid sessions are unaffected — your application validates their token locally using the cached JWKS public key. Only new login attempts require a live Invysta connection. With HA deployment, planned maintenance causes zero downtime.

11. Security Model

Property	Detail
End-to-end payload confidentiality	Device fingerprint and AAK data is encrypted by the Invysta client app using the Invysta service public key (RSA-OAEP + AES-GCM) before leaving the device. The payload is never exposed in plaintext to your platform, network infrastructure, or any intermediary. Only the Invysta service, holding the private key, can decrypt it.
No credential stored at Invysta	The trust anchor is the device fingerprint, not a reusable credential. There is no syncable password or token object for an attacker to steal from Invysta infrastructure.
Device-bound authentication	Authentication is tied to a registered physical device. A stolen password alone cannot grant access — the device fingerprint must also match.
Standard OIDC token output	Despite the proprietary device verification happening inside the interaction, the output is a standard JWT. Your application validates tokens using the published JWKS public key — no Invysta-specific library required.
Session token validation is local	Your application caches the Invysta JWKS public key and validates tokens locally. An Invysta service interruption does not affect users with existing valid sessions.
All logic server-side at Invysta	Device fingerprint matching, AAK verification, and cryptographic operations run entirely within Invysta infrastructure. Your codebase contains no authentication logic beyond the standard OIDC redirect and token exchange.

12. Integration Checklist



Receive client credentials from Invysta

client_id, optional client_secret, and your dedicated provider URL.



Ensure end users enroll their devices

Each user installs the Invysta mobile app and completes one-time device enrollment before their first login.

Enrollment is capped at the licensed maxAuthorizedDevices per user.



Point your OIDC library at the discovery URL

One line of config in Passport.js, openid-client, Spring Security, .NET OIDC middleware, etc.



Add a callback route to your app

Handle the /auth/callback redirect. Verify state, exchange the code, validate the id_token.



Test with OidcDebugger

Use <https://oidcdebugger.com> to verify the full flow before wiring into your app. Zero setup required.



Go live

Switch redirect_uri to your production URL. No other changes required.

No Invysta SDK. No agent to install. No changes to your data model.